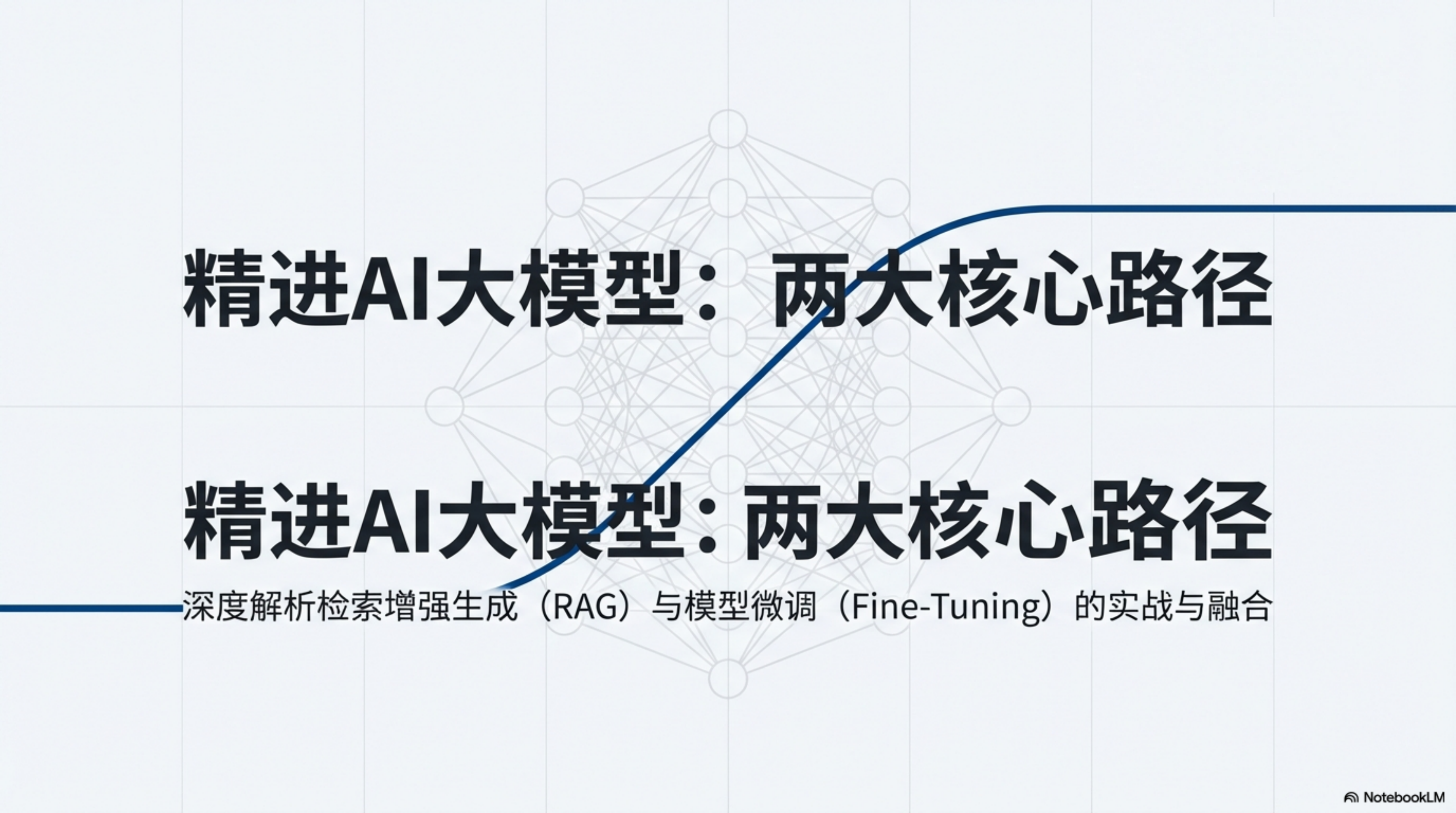




精进AI大模型：两大核心路径

精进AI大模型：两大核心路径

深度解析检索增强生成（RAG）与模型微调（Fine-Tuning）的实战与融合

超越基础模型：通往卓越AI的两条精进之路

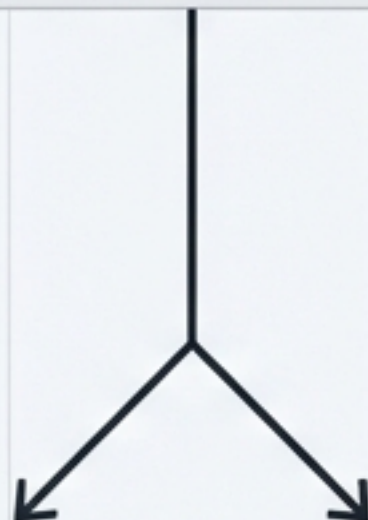
基础LLM存在固有局限：

- 知识截止：无法获取最新信息。
- 内容幻觉：可能生成不准确或虚构的内容。
- 风格泛化：输出格式和风格通用，缺乏特定领域的专业性。



知识之路：RAG（外挂知识）

在运行时为模型注入动态、精准的外部信息。

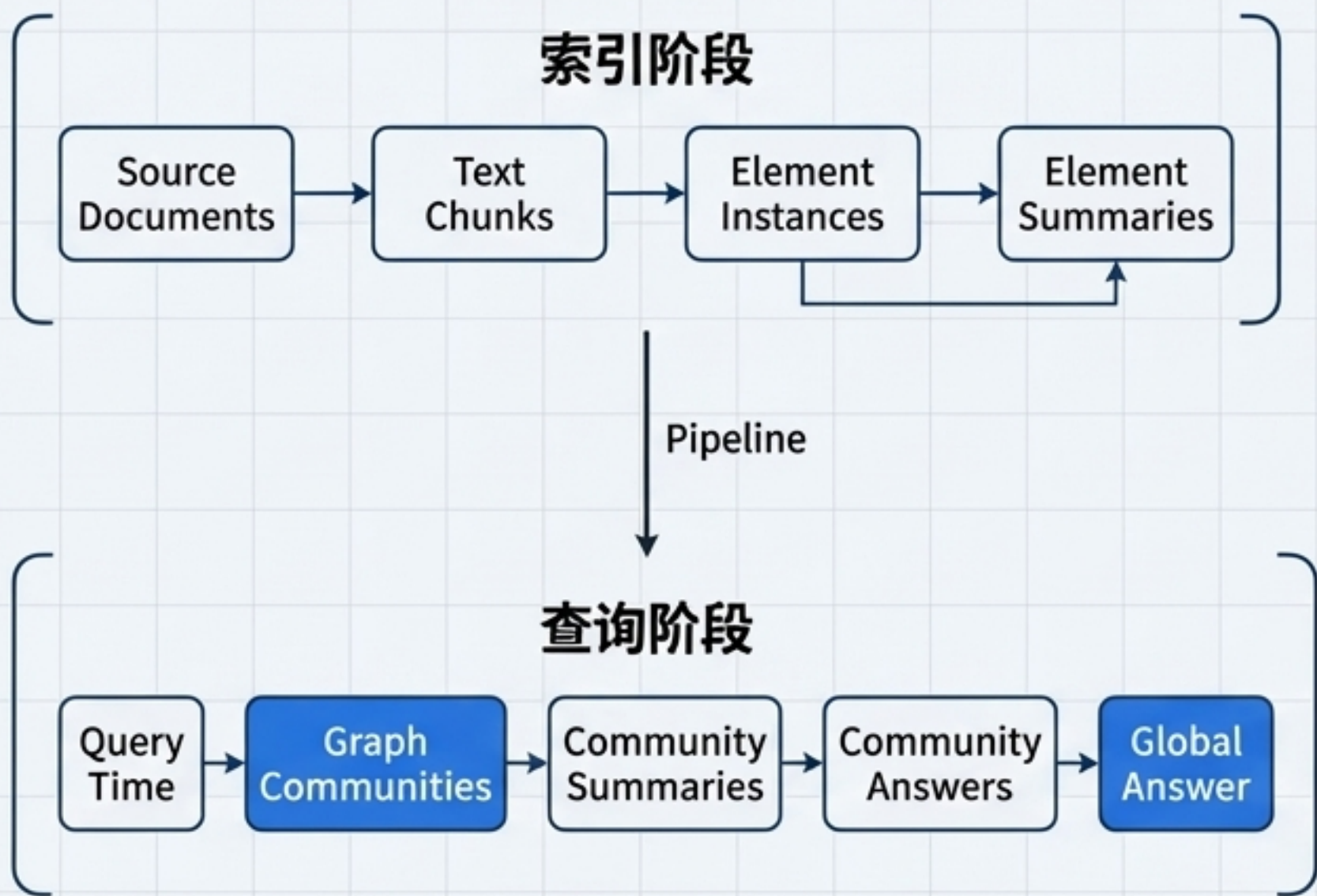


能力之路：微调（内化能力）

塑造模型的核心行为、风格和遵循特定指令的能力。

GraphRAG：从局部到全局的图驱动式摘要

GraphRAG结合了LLM和图索引的优势，以解决大型文本语料库中以查询为中心的摘要挑战。



关键流程解读

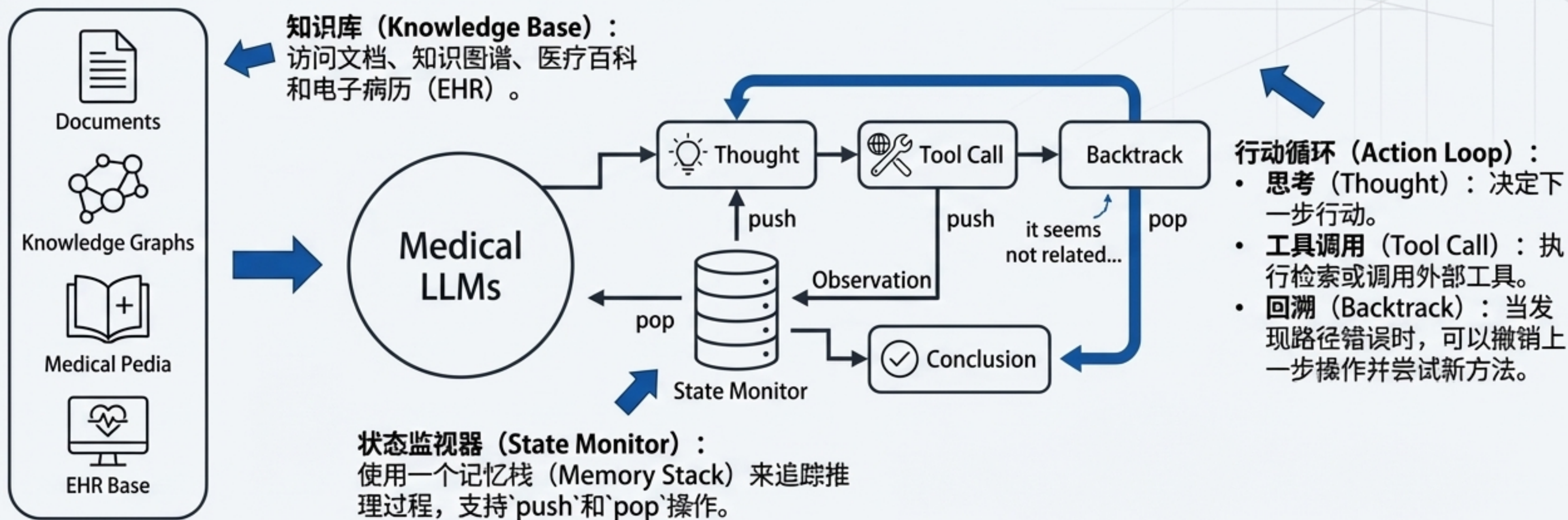
1. **索引阶段 (Indexing Stage)**：提取源文档，构建包含实体、关系和声明的知识图谱。
2. **查询阶段 (Query Stage)**：在图上进行查询，利用社区检测来定位相关信息集群。
3. **生成 (Generation)**：对图社区进行分层摘要，最终生成一个全局答案。

关键启示

RAG正从简单的向量检索，进化为对结构化知识进行深度理解和推理。

RAG的演化：迈向图灵完备的推理系统

TC-RAG：具备思考、回溯与工具调用能力的医疗LLM系统



应用场景： 引用医生与系统的交互来说明其智能。例如，当检索结果不理想时，系统可以像人类专家一样‘换一个工具或者换一个查询方式，并删除历史检索中可能的错误’。

能力之路：通过微调内化模型的核心技能

微调并非注入新知识，而是教会模型如何“说”与“做”——即复制特定的结构、风格或格式。

效果对比：医疗诊断意见生成

基础模型输出

患者被诊断为冠状动脉疾病，需要进行冠状动脉旁路移植术。术后需要一段时间的休养，期间需要服用一些药物，如阿司匹林和降胆固醇药物。医生建议患者避免剧烈运动，并定期进行心电图检查。



微调后模型输出

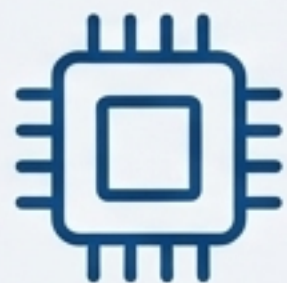
患者被诊断为**冠状动脉疾病**（CAD），需进行**冠状动脉旁路移植术**（CABG），术后恢复期建议患者遵循严格的康复计划，包括适度的运动康复和药物治疗。主要包括**阿司匹林**、 **β 受体阻滞剂**和**他汀类药物**，以防止血栓形成和降低胆固醇水平。医生建议患者在术后前三个月避免任何剧烈的身体活动，并定期进行心电图检查。

⊖ 评价：准确但泛化，缺乏专业术语和结构。

⊕ 评价：更专业、结构化，并使用了行业标准缩写（CAD，CABG）。

实战演练：Llama-3 8B模型微调指南

*通过一个完整的代码示例，演示如何使用PEFT（参数高效微调）技术对Llama-3模型进行训练，以遵循特定的指令格式。



模型 (Model)

unsloth/llama-3-8b-bnb-4bit

关键技术：使用`unsloth`库进行优化，通过4-bit量化加载模型，极大减少显存占用。



框架 (Framework)

PyTorch, transformers, peft, bitsandbytes



数据 (Data)

Alpaca格式的指令数据集（.json`格式）。



环境 (Environment)

Google Colab.

步骤一：准备高质量的指令数据集

“微调的成功始于数据。模型将学习复制你所提供数据的格式和风格。”

数据格式：Alpaca

一种简单而强大的JSON结构，包含三个关键字段。

· **instruction**：模型需要执行的任务描述。

· **input**：（可选）任务的上下文或输入。

· **output**：期望模型生成的目标回答。

cuosth.json

```
[
  {
    "instruction": "只剩一个心脏了还能活吗？",
    "input": "",
    "output": "能，人本来就只有一个心脏。",
  },
  {
    "instruction": "爸爸再婚，我是不是就有了个新娘？",
    "input": "",
    "output": "不是的，你有了个继母。\\\"新娘\\\"是指新婚的女方，而你爸爸再婚，他的新婚妻子对你来说是继母。",
  },
  ...
]
```


步骤二与三：加载模型与构建指令模板

加载4位量化模型

```
# 1. 导入FastLanguageModel
from unsloth import FastLanguageModel
import torch

# 2. 设置模型参数
max_seq_length = 2048
dtype = None # None for auto-detection
load_in_4bit = True

# 3. 从HuggingFace加载模型
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name = "unsloth/llama-3-8b-bnb-4bit",
    max_seq_length = max_seq_length,
    dtype = dtype,
    load_in_4bit = load_in_4bit,
)
```

→ **load_in_4bit = True**：这是实现高效微调的关键，它让强大的8B模型能在消费级GPU上运行。

定义Alpaca指令格式

```
# 这是模型在训练和推理时看到的文本结构
alpaca_prompt = """Below is an instruction that
describes a task, paired with an input that provides
further context. Write a response that appropriately
completes the request.

### Instruction:
{}

### Input:
{}

### Response:
{}"""
```

→ **alpaca_prompt**：定义了一个严格的模板。微调过程就是教会模型理解这个模板，并在看到**### Response:**后生成符合期望的内容。

步骤四与五：配置PEFT（LoRA）并启动训练

核心技术：PEFT（LoRA）

我们不训练模型的全部参数，而是只训练一小部分“适配器”（Adapter）层。这使得训练速度更快，资源消耗更少，同时保留了原始模型的绝大部分能力。

1. 应用PEFT配置

```
from trl import SFTTrainer
from transformers import TrainingArguments

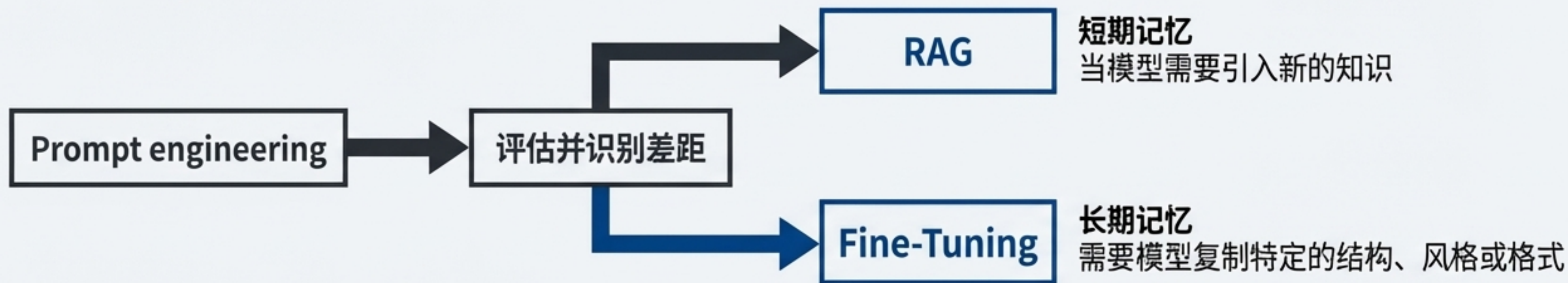
# 将LoRA适配器应用到模型
model = FastLanguageModel.get_peft_model(
    model,
    r = 16, # LoRA rank
    target_modules = ["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj",],
    lora_alpha = 16,
    lora_dropout = 0,
    bias = "none",
    use_gradient_checkpointing = True,
)
```

2. 设置训练参数并执行

```
trainer = SFTTrainer(
    model = model,
    tokenizer = tokenizer,
    train_dataset = dataset,
    dataset_text_field = "text",
    max_seq_length = max_seq_length,
    args = TrainingArguments(
        per_device_train_batch_size = 2,
        gradient_accumulation_steps = 4,
        learning_rate = 2e-4,
        # ... 其他参数 ...
        output_dir = "outputs",
    ),
)

# 开始训练!
trainer.train()
```


知识 vs. 能力：如何选择与融合



何时选择 RAG?

场景: 短期记忆 (Short-term Memory)

需求: 当模型需要引入新的、动态变化的、或专有的知识时。

例子: 回答关于今天新闻的问题；查询公司内部文档；基于用户最新的聊天记录进行回复。

何时选择 Fine-Tuning?

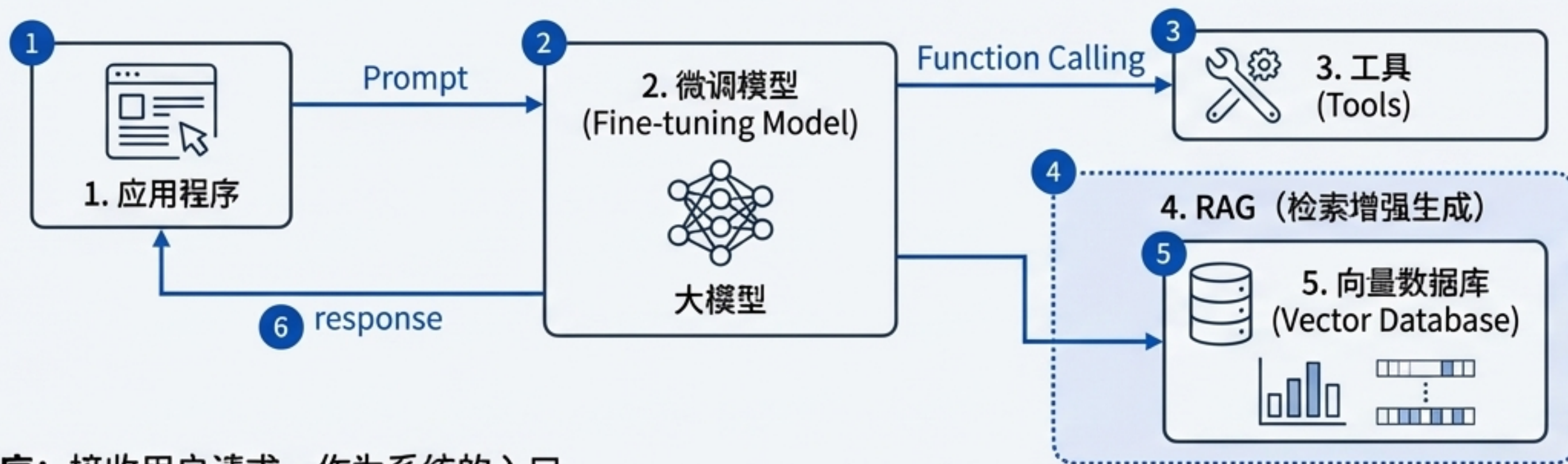
场景: 长期记忆 (Long-term Memory)

需求: 当需要模型复制特定的结构、风格、格式或学习新的行为模式时。

例子: 生成特定格式的JSON输出；模仿特定人物的说话风格；遵循复杂的多步骤指令。

“这些都是附加的，而不是独有的。对于某些场景，两者叠加以获得最佳性能。”

融合架构：构建现代大模型应用



- 1. 应用程序：**接收用户请求，作为系统的入口。
- 2. 微调模型：**这是系统的“大脑”。它经过微调，擅长理解用户意图、进行函数调用（Function Calling）和生成结构化响应。
- 3. 工具：**当需要与外部世界交互时（如查询API、执行代码），模型会调用预定义的工具。
- 4. RAG：**当模型判断需要外部知识来回答问题时，它会触发RAG流程。
- 5. 向量数据库：**RAG模块在这里通过Embedding检索最相关的知识片段。
- 6. 响应：**微调模型整合来自RAG和工具的信息，最终生成一个精准、丰富的响应返回给应用程序。

关键洞察：在专业领域（如医学、法律、金融），一个仅有基础能力的模型是不足的。一个经过微调以理解领域任务、并能通过RAG访问领域知识的系统，才是真正可用的解决方案。

人工智能的未来：构建 AI Agent

RAG与微调的融合不仅是当前构建强大应用的最佳实践，更是通往真正AI Agent的基石。

核心推理引擎 (Core Reasoning Engine)



一个经过深度**微调**的模型，使其具备强大的规划、决策和工具使用能力。

动态知识系统 (Dynamic Knowledge System)



一个先进的**RAG**系统，使其能够实时感知环境、获取新信息并适应变化。

行动与执行 (Action & Execution)



通过调用工具和API与数字或物理世界进行交互的能力。

我们今天探讨的两条精进之路——赋予模型外部知识和内化核心能力——最终将在AI Agent的形态上汇合，开启人工智能的新篇章。
